Recursion, Recurrences, and Divide-and-Conquer

A recursive call has a contract. We must explain what it returns, why its subcalls finish, and how their work combines with the work of the current call.

Read in this order: **one recursive path → merge sort → tools for proving recurrence bounds → maximum subarrays and matrix multiplication → unequal-split extensions**.

Prerequisites: Chapter 1's invariants, induction, asymptotic notation, and sums. Matrix examples use elementary matrix multiplication; the extension uses integration.

Reference textbook: Cormen et al., *Introduction to Algorithms*, 3rd Edition, Chapters 2 and 4; supplementary teaching reference: MIT 6.046J (2005), Lectures 2–3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A subcall contract is the interface for induction

Let $P(n)$ mean: for every legal input of size n, the procedure terminates and returns a result satisfying its specification.

1. **Base cases:** check the smallest inputs, including whether empty input is legal.
2. **Shrinking:** every subproblem is legal and strictly smaller than n.
3. **Combination:** assuming the smaller calls are correct, prove the current result correct.

This is strong induction. Assuming the current call is correct would be circular. Shrinking proves termination, not the result specification by itself.

Analyze cost separately: $T(n) = \sum_i T(n_i) + f(n)$. Charge only the current call's comparisons, partitioning, copies, and combination to f; **do not charge a subtree twice**.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary search continues down one branch

Preconditions: $A[1..N]$ is nondecreasing over totally ordered keys. Return **any** matching index, or NIL if none exists. Call with $(1, N)$; $N = 0$ returns immediately.

```
BINARY-SEARCH(A, low, high, x)
    if low > high: return NIL
    mid = low + floor((high-low)/2)
    if A[mid] == x: return mid
    if A[mid] < x:
        return BINARY-SEARCH(A, mid+1, high, x)
    return BINARY-SEARCH(A, low, mid-1, x)
```

Intervals include both endpoints. A nonempty call has $1 \le low \le high \le N$; an empty interval reads no element. This version does not promise the first duplicate.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Why may an entire interval be discarded?

If $A[mid] < x$, then $A[i] \le A[mid] < x$ for every $i \le mid$. Any match must lie on the right. If $A[mid] > x$, keep only the left. Equality already satisfies the contract.

For current length $n = high - low + 1 \ge 1$, a recursive interval has length at most $\lfloor n/2 \rfloor < n$. The empty interval is the failure base case. Strong induction on length proves termination and correctness.

With constant-cost comparisons and index-only arguments, worst-case cost satisfies

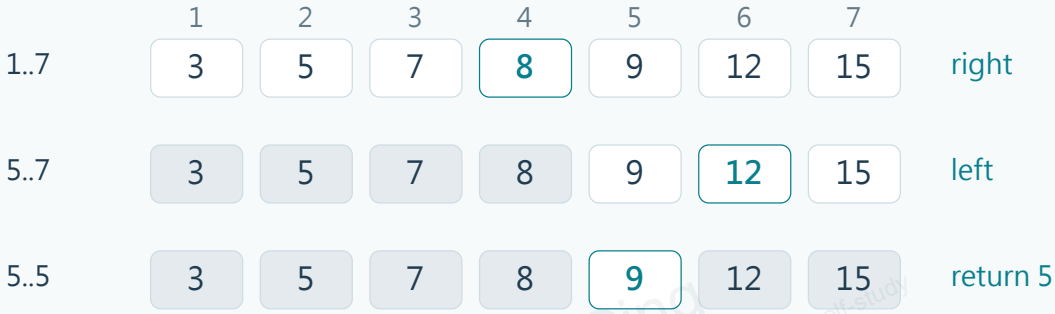
$$W(0) = \Theta(1), \qquad W(n) = W(\lfloor n/2 \rfloor) + \Theta(1).$$

The longest path has $\Theta(\lg(n+1))$ calls; a match at the root costs $\Theta(1)$. Recursive execution has peak stack space $\Theta(\lg(n+1))$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Trace both success and failure on the same array

Find 9: white cells remain; teal border marks the probe



For 10: call the empty interval 6..5 next → NIL.

For $A = [3, 5, 7, 8, 9, 12, 15]$, searching for 9 checks $(low, mid, high) = (1, 4, 7), (5, 6, 7), (5, 5, 5)$ and returns 5.

Searching for 10 makes the same first three probes. Since $9 < 10$, it then calls $(6, 5)$ and returns NIL on the empty interval. Gray cells are excluded candidates; the array itself is unchanged.

Copying can change the cost of the same search logic

Keep original length N distinct from current interval length n. On a worst-case search path:

Argument representation	Local work per call	Total worst-case cost
Indices or a view	$\Theta(1)$	$\Theta(\lg(N+1))$
Copy the current interval	$\Theta(n)$	$\Theta(N)$
Copy the original whole array	$\Theta(N)$	$\Theta(N \lg(N+1))$

The second row sums $N + N/2 + N/4 + \cdots = \Theta(N)$. In the third row, N stays fixed throughout recursion; replacing it by n changes the model.

A recursion tree may have only one child per node. It then becomes a path. Multiple recursive calls are not a prerequisite for the method.

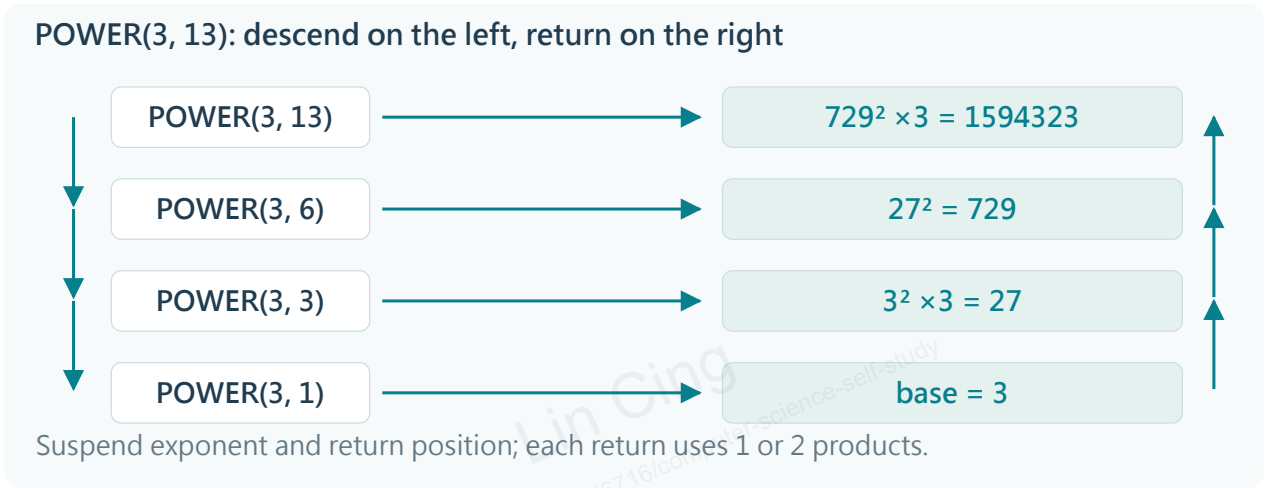
Repeated squaring: compute the subproblem once

Input is a nonnegative integer n; output is a^n, with $a^0 = 1$. We first count multiplications; this does not make arbitrary-precision multiplication constant-time.

```
POWER(a, n)
    if n == 0: return 1
    if n == 1: return a
    y = POWER(a, floor(n/2))
    z = y*y
    if n is odd: z = z*a
    return z
```

If $n = 2k$, the subcall returns $y = a^k$, so $y^2 = a^n$. If $n = 2k + 1$, multiply once more by a. The two bases and $\lfloor n/2 \rfloor < n$ complete strong induction.

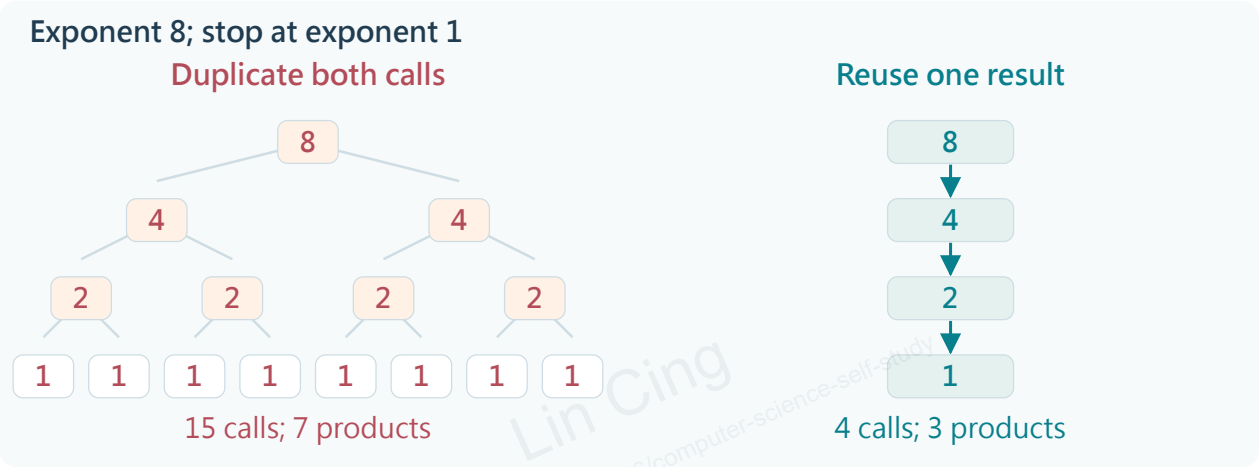
Descend with suspended calls, then compute on return



For 3^{13}, exponents descend as $13 \to 6 \to 3 \to 1$. Returned values are $3 \to 27 \to 729 \to 1594323$, using 0, 2, 1, and 2 multiplications: five in total.

For $n \geq 1$, there are $\lfloor \lg n \rfloor$ squarings and $\operatorname{popcount}(n) - 1$ extra multiplications by a. Total multiplications are their sum. At $n = 0$, the count is zero.

Two occurrences in an identity need not mean two calls



Writing two separate POWER calls for $a^{n/2}a^{n/2}$ gives $D(n) = 2D(n/2) + \Theta(1) = \Theta(n)$ on powers of two, despite equal returned values.

Saving one returned value before squaring gives $R(n) = R(\lfloor n/2 \rfloor) + \Theta(1) = \Theta(\lg n)$ under unit-cost multiplication. Count **executed calls**, not occurrences left after algebraic simplification.

Fixed-width code must handle multiplication overflow. Modular exponentiation may reduce after each product, but the intermediate product must itself be computed safely.

Fibonacci: repeated calls and the size of the answer

Define $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$. Naive recursion repeats work. Its call count obeys $C_0 = C_1 = 1$ and $C_n = C_{n-1} + C_{n-2} + 1$.

Setting $D_n = C_n + 1$ proves $C_n = 2F_{n+1} - 1$ from the recurrence and bases. Computing $F_5 = 5$ takes 15 calls. Time is $\Theta(\varphi^n)$ but stack depth is only $\Theta(n)$, where $\varphi = (1 + \sqrt{5})/2$.

Computing in increasing order while retaining the previous two values takes $n - 1$ additions for $n \geq 2$. This introduces reuse; full dynamic-programming design comes later.

Separate arithmetic steps from bit complexity. The answer has $\Theta(n)$ bits. Two retained integers need not occupy a constant number of machine words. Sequential ordinary integer additions take $\Theta(n^2)$ bit operations.

Compute Fibonacci exactly through a matrix power

Let $Q = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$. For $n \geq 1$,

$$Q^n = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}.$$

The identity holds at $n = 1$. Right multiplication by Q adds the two old columns to form the first new column and copies the first old column to the second. The Fibonacci recurrence proves the next case.

For example, $Q^2 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$ and $Q^4 = \begin{bmatrix} 5 & 3 \\ 3 & 2 \end{bmatrix}$, so $Q^5 = \begin{bmatrix} 8 & 5 \\ 5 & 3 \end{bmatrix}$ gives $F_5 = 5$. Return 0 directly at $n = 0$.

Repeated squaring needs $O(\lg(n+1))$ exact 2×2 matrix multiplications. This counts **arithmetic operations**. Writing the $\Theta(n)$-bit answer already rules out logarithmic bit time. Floating-point evaluation of the closed form is not an unconditional exact algorithm.

Merge Sort and Divide-and-Conquer

Divide and conquer solves strictly smaller instances and combines their results. State boundaries, base cases, merge preconditions, and results. For $A[p..r]$, length is $m = r - p + 1$, with $p > r$ denoting an empty interval.

```
MERGE-SORT(A,p,r)
    if p >= r: return
    q = p + floor((r-p)/2)
    MERGE-SORT(A,p,q)
    MERGE-SORT(A,q+1,r)
    MERGE(A,p,q,r)
```

A legal nonempty call has $1 \leq p \leq r \leq n$. The full call is $(1, n)$, giving immediate return at $(1, 0)$ for empty input. Child lengths are $\lceil m/2 \rceil$ and $\lfloor m/2 \rfloor$, both smaller for $m \geq 2$. Computing the midpoint from a difference avoids the usual $p + r$ overflow, provided the index range itself is representable.

MERGE without an extra key value

Assume $1 \le p \le q < r \le n$ and two stably sorted halves. L and R are **independent copies**, not views affected by overwriting A. Their lengths are $q - p + 1$ and $r - q$.

```
MERGE(A,p,q,r)
    L = copy(A[p..q]); R = copy(A[q+1..r])
    i = 1; j = 1; k = p
    while i <= L.length and j <= R.length
        if L[i].key <= R[j].key: A[k] = L[i]; i = i+1
        else: A[k] = R[j]; j = j+1
        k = k+1
    while i <= L.length
        A[k] = L[i]; i = i+1; k = k+1
    while j <= R.length
        A[k] = R[j]; j = j+1; k = k+1
```

Choose L on equality. Every original left-half record precedes every right-half record, so stability is preserved. Length checks require no key-domain value named infinity.

Proof of merge preservation, order, and termination

Before the main loop,

$$k - p = (i - 1) + (j - 1).$$

The output $A[p..k-1]$ is exactly the stable merge of consumed prefixes $L[1..i-1], R[1..j-1]$. It is sorted, and every emitted key is at most every unconsumed key. Remaining records still exist in independent buffers.

The empty output establishes initialization. When both inputs remain nonempty, their heads are their smallest remaining records. Choosing the smaller head, with left priority on ties, gives the next record of the stable order. Consuming one record and incrementing k preserves lengths, data, order, and stability. Remaining record count decreases by one, proving termination.

At main-loop exit, at least one buffer is exhausted. The other is sorted with keys no smaller than the output tail, so appending it preserves the invariant. Finally $i = |L| + 1, j = |R| + 1, k = r + 1$, and all m records occur once. Without copies, an output write could destroy unread input and invalidate the proof.

Complete merge with duplicate keys

Let $L = [1a, 4b, 4c]$, $R = [2d, 4e, 6f]$, with L originally before R. Letters label records; compare only numeric keys.

i,j before test	Comparison	Output prefix
1,1	$1 \leq 2$; take 1a	1a
2,1	$4 > 2$; take 2d	1a,2d
2,2	$4 \leq 4$; take left 4b	1a,2d,4b
3,2	$4 \leq 4$; take left 4c	1a,2d,4b,4c
4,2	L exhausted; no key comparison	append 4e,6f

There are 4 key comparisons, 6 output writes, and 6 copied buffer records. Interleaving until the last two records can require $m - 1 = 5$ comparisons. $\Theta(m)$ work does not mean exactly m key comparisons.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete merge with duplicate keys — diagram

Take left on equality:4b and 4c must precede 4e

	L			R			Output prefix
start	1a	4b	4c	2d	4e	6f	ø
1≤2	1a	4b	4c	2d	4e	6f	1a
4>2	1a	4b	4c	2d	4e	6f	1a, 2d
4≤4	1a	4b	4c	2d	4e	6f	1a, 2d, 4b
4≤4	1a	4b	4c	2d	4e	6f	1a, 2d, 4b, 4c
tail	1a	4b	4c	2d	4e	6f	1a, 2d, 4b, 4c, 4e, 6f

Tinted cells are consumed; unconsumed records remain in independent copies.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Sentinel preconditions

A sentinel implementation can append an infinity record to each buffer and select the smaller head exactly m times, avoiding repeated exhaustion tests. The sentinel key must strictly exceed every legal key, and its record must not be legal input. The maximum legal integer is unsafe as a sentinel.

If legal keys include $+\infty$, take $L = [+\infty]$, $R = [+\infty, +\infty]$. Left priority can select the real left record, then its equal-valued sentinel, then read beyond L on the third output. This is not an out-of-domain marker.

Length checks work for any totally ordered key domain. A tagged sentinel is another option, but it must rank strictly above every real record and never be emitted as data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From merge to complete sorting

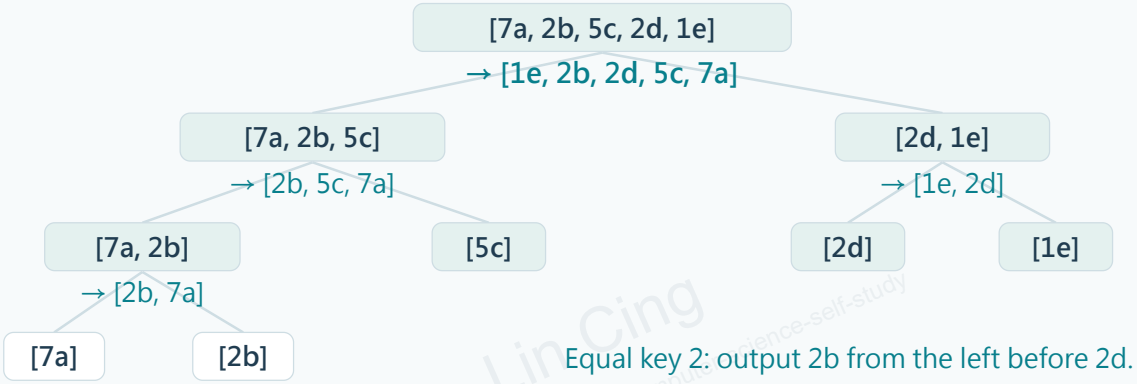
Use strong induction on interval length m. At $m \leq 1$, input is sorted and unchanged. For $m \geq 2$, both smaller calls stably sort and preserve their records by induction. MERGE's preconditions hold, and its proof gives a stable sorting of the whole interval, leaving outside storage unchanged. Child sizes decrease to 0 or 1, proving termination without a power-of-two assumption.

For [7a,2b,5c,2d,1e], splits have lengths 3 and 2, then 2/1 and 1/1. The figure shows every merged node. At the root tie, left 2b still precedes right 2d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From merge to complete sorting — diagram

Divide downward; labels show each returned sorted result



Equal key 2: output 2b from the left before 2d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Include merging in the recurrence

Midpoint computation costs $\Theta(1)$. Copying and outputting m records costs $\Theta(m)$, giving

$$T(m) = T(\lceil m/2 \rceil) + T(\lfloor m/2 \rfloor) + \Theta(m), \quad T(0), T(1) = \Theta(1).$$

For $m = 2^h$, first solve the exact surrogate $U(1) = d$, $U(m) = 2U(m/2) + cm$, with positive c,d. Depth $\ell < h$ has 2^ℓ nodes costing $cm/2^\ell$ each, hence cm per level. There are h internal levels and m leaves of cost d, so $U(m) = cm \lg m + dm$.

Actual internal costs lie between fixed positive multiples of size, and leaves have constant lower and upper costs. Apply the expansion to both bounds to prove Θ(m log m), rather than assuming one exact c describes every instruction and base case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Arbitrary lengths and a six-record cost table

At depth ℓ, each surviving interval has size at most $\lceil m/2^\ell \rceil$, so height is at most $\lceil \lg m \rceil$. Intervals at one depth are disjoint with total size at most m, proving O(m log m).

At depths $\ell < \lfloor \lg m \rfloor$, each interval has size at least $\lfloor m/2^\ell \rfloor \geq 2$; none has become a leaf. Total size is exactly m, giving Ω(m) at each of $\lfloor \lg m \rfloor$ levels. For $m \geq 2$, this is at least $(\lg m)/2$, proving the lower bound for arbitrary m.

Depth for m=6	Sizes	Total internal size
0	6	6
1	3,3	6
2	2,1,2,1	4
3	1,1,1,1; two earlier leaves	0

Buffer copies and output writes each total 16, not 6 at every depth. There are six base cases.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Peak space differs from cumulative allocation

MERGE allocates L/R only after both recursive calls return and releases them after merging. A length-m merge holds m buffer records. Child buffers do not coexist with parent buffers, giving $\Theta(n)$ peak auxiliary record space plus $\Theta(\lg n)$ stack space, hence $\Theta(n)$ overall for $n \geq 2$.

Cumulative allocation may be $\Theta(n \lg n)$, but it is not simultaneous demand. Retaining completed buffers changes the lifetime model and invalidates this peak bound. A reusable n-record scratch buffer is also possible with the same nonaliasing requirement.

Insertion sort needs only a saved record and constant index words. This merge sort guarantees $\Theta(n \lg n)$ time but is not in-place. Both are stable under their respective strict-shift and left-tie rules.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Substitution: prove one fixed bound

Guess a bound, then use strong induction with the same fixed constants at every size.

Let $T(1) = 1$ and $T(n) = 2T(\lfloor n/2 \rfloor) + n$ for $n \geq 2$. Try $T(n) \leq cn \lg n$. For $n \geq 4$, put $m = \lfloor n/2 \rfloor \geq 2$. Then

$$\begin{aligned} T(n) &\leq 2cm \lg m + n \\ &\leq cn \lg(n/2) + n \\ &= cn \lg n - (c-1)n \leq cn \lg n \end{aligned}$$

for fixed $c \geq 1$. At $n = 1$ the candidate right side is zero, so that input cannot establish this induction. Instead, compute $T(2) = 4, T(3) = 5$ from the recurrence. Fixed $c = 2$ covers both, and every child at $n \geq 4$ is at least 2, closing induction.

A recurrence base and an induction base may differ. The proof must still cover every reached small input.

Lower bounds and stronger hypotheses

First restrict to powers of two, with $T(1) = 1$ and $T(n) = 2T(n/2) + n$. Assuming $T(n/2) \geq d(n/2) \lg(n/2)$ gives

$$T(n) \geq dn \lg n + (1-d)n \geq dn \lg n, \qquad 0 < d \leq 1.$$

The base holds. To extend this to every integer, first prove T is nondecreasing: $T(2) > T(1)$; for $n \geq 2$, strong induction gives $T(n+1) - T(n) = 2[T(\lfloor (n+1)/2 \rfloor) - T(\lfloor n/2 \rfloor)] + 1 \geq 1$. Set $p = 2^{\lfloor \lg n \rfloor}$; then $p > n/2$ and $1 + \lg p > \lg n$. Since $T(p) = p(1 + \lg p)$, monotonicity yields $T(n) \geq T(p) > \frac{1}{2} n \lg n$.

Now consider $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$, with $T(1) = 1$. The guess $T(n) \leq cn$ yields only $cn + 1$. Strengthen it to

$$T(n) \leq cn - d.$$

Substitution yields $T(n) \leq cn - 2d + 1 \leq cn - d$ for $d \geq 1$. Choosing $c = 2, d = 1$ covers the base; induction in fact gives the exact answer $2n - 1$.

The lower-order deductions at children pay the new work at the parent. Saying that $cn + n$ is still $O(n)$ does not establish the same fixed bound cn and cannot close induction.

Variable changes and strict shrinking

Start on $n = 2^{2^k}$ with $T(n) = 2T(\sqrt{n}) + \lg n$ and $T(2) = \Theta(1)$. Set $m = \lg n$ and $S(m) = T(2^m)$. Then

$$S(m) = 2S(m/2) + m = \Theta(m \lg m).$$

Restoring the variable gives $T(n) = \Theta(\lg n \lg \lg n)$, not a bound with the new variable mistaken for the original input. The next argument treats an integer version with a floor.

A fixed shift also needs a shrinking check. In $T(n) = 2T(n/2 + 17) + n$, children shrink only for $n > 34$. Put $n = 34 + x$ and $U(x) = T(34 + x)$, obtaining

$$U(x) = 2U(x/2) + x + 34.$$

With a sufficiently large fixed base interval, level sums give $\Theta(x \lg x) = \Theta(n \lg n)$. This argument first uses real arguments; integer versions must also bound rounding. Giving only $T(1)$ would leave $n = 34$ calling itself, so the recurrence would not be well-defined.

How to justify a floor inside a square root

Define $T(n) = 1$ for integers $1 \le n \le 3$, and for $n \ge 4$ let

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n.$$

Here $2 \le \lfloor \sqrt{n} \rfloor < n$, so the recurrence is well-defined and terminates.

Set $a_k = 2^{2^k}$ and $U_k = T(a_k)$. Since $\sqrt{a_k} = a_{k-1}$, these points have no rounding error. We have $U_0 = 1$ and $U_k = 2U_{k-1} + 2^k$.

Dividing by 2^k and summing yields $U_k = (k+1)2^k$. Thus at these special inputs, $T(n) = \lg n(1 + \lg \lg n)$.

This establishes a result only on the special sequence. Extending it to every integer requires the monotonicity and sandwich argument next.

Prove monotonicity before sandwiching inputs

The bases satisfy $T(1) = T(2) = T(3) = 1 < T(4)$. Assume smaller-input values are nondecreasing. Since $\lfloor \sqrt{n+1} \rfloor \geq \lfloor \sqrt{n} \rfloor$, both child arguments are below n, and $\lg(n+1) > \lg n$, we get $T(n+1) \geq T(n)$. Strong induction proves monotonicity.

Choose k with $a_k \leq n < a_{k+1}$, equivalently $2^k \leq \lg n < 2^{k+1}$. Monotonicity gives

$$(k+1)2^k \leq T(n) \leq (k+2)2^{k+1}.$$

Within this interval, $\lg n$ is within a factor two of 2^k, and $1 + \lg\lg n$ is within fixed factors of $k+1$. Therefore

$$T(n) = \Theta\big(\lg n(1 + \lg\lg n)\big), \qquad n \geq 2.$$

For sufficiently large n, this is $\Theta(\lg n \lg\lg n)$. The sandwich uses proved monotonicity, not an assumption that the floor can be discarded.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A recursion tree charges local work once

Each internal node stores only its current nonrecursive work $f(\text{size})$; its descendants account for recursive work. Storing the full $T(\text{size})$ at a node and also summing its descendants would double-count.

Take $T(n) = 3T(n/4) + n^2$, $T(1) = 1$, initially on $n = 4^L$. Depth j has 3^j nodes, each costing $(n/4^j)^2$, so its total is $n^2(3/16)^j$. There are $3^L = n^{\log_4 3}$ leaves. Thus

$$T(n) = n^2 \sum_{j=0}^{L-1} (3/16)^j + n^{\log_4 3}.$$

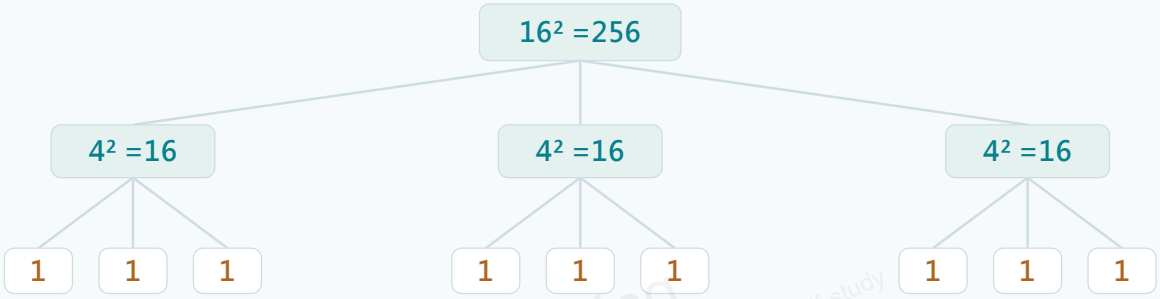
The geometric sum is at most $16/13$, and leaf work is at most n^2, proving $O(n^2)$. Root work supplies the matching $\Omega(n^2)$ lower bound.

At $n = 16$, level costs are 256,48,9, totaling 313. The final 9 already pays for nine base cases; do not add $f(1)$ again at each leaf.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A recursion tree charges local work once — diagram

T(n)=3T(n/4)+n², T(1)=1; n=16



Level costs:256 →48 →9 (nine base calls)
Total 313; do not add f(1) again at leaves.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A contracting level sum still needs leaf costs

For integer $n \geq 8$, define

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lfloor n/4 \rfloor) + T(\lfloor n/8 \rfloor) + n,$$

with $T(n) = 1$ for $0 \leq n \leq 7$. Child sizes sum to at most $7n/8$. Internal work at depth j is at most $n(7/8)^j$, totaling at most $8n$.

Let I and L count internal nodes and leaves. Every internal node has three children, so counting edges gives $3I = I + L - 1$ and $L = 2I + 1$. Each internal node costs at least 8, hence $8I \leq 8n$ and $L \leq 2n + 1$.

Thus $T(n) \leq 8n + 2n + 1 = O(n)$. The root supplies $T(n) \geq n$, proving $\Theta(n)$. **The geometric sum pays for internal work; a separate node count pays for leaves.**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unequal splits: depth and mass per level

Let $a = \lfloor n/3 \rfloor$, $b = n - a$, and $T(n) = T(a) + T(b) + cn$ for fixed $c > 0$, with fixed positive bounds on small-input costs.

Along any path, the depth-j size q_j satisfies

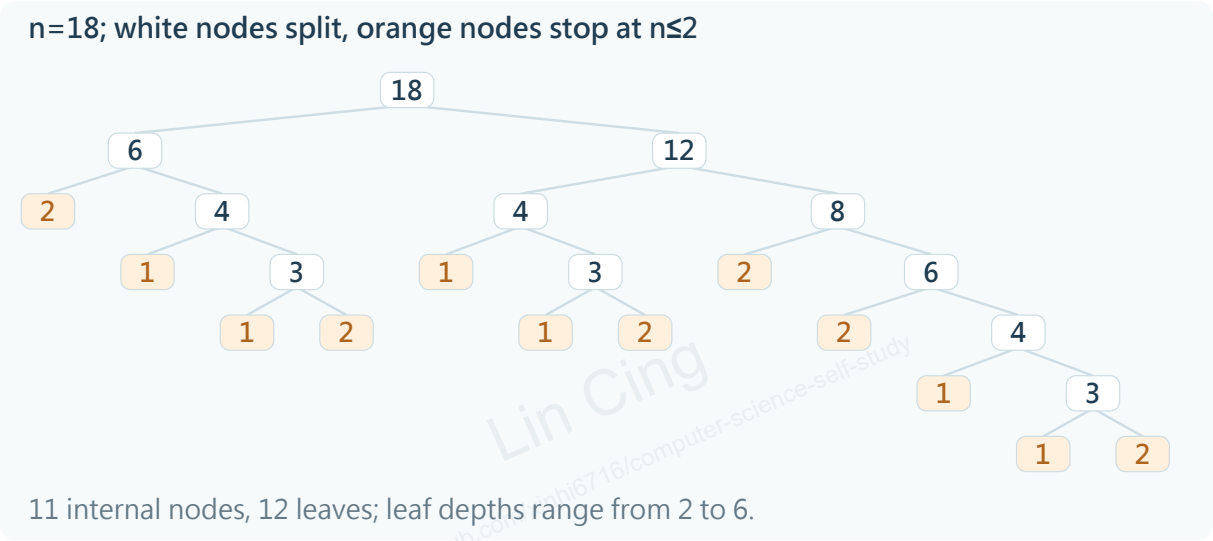
$$\frac{n}{3^j} - \frac{3}{2} \le q_j \le n(2/3)^j + 3.$$

Induct from $q/3 - 1 \le q_{\text{next}} \le 2q/3 + 1$ and sum the geometric rounding errors. Absorb all work at sizes up to fixed $K \ge 6$ into base cases; this changes only constants. The upper bound gives $O(\lg n)$ height. Disjoint intervals at a depth have total length at most n and cost at most cn. Positive integer leaf lengths sum to n, so there are at most n leaves. Total cost is $O(n \lg n)$.

For $0 \le j \le \lfloor \log_3(n/(K+2)) \rfloor$, the lower bound keeps every $q_j > K$. No path has stopped: each complete level retains mass n and costs cn. There are $\Omega(\lg n)$ such levels, proving the matching lower bound.

With only $O(n)$ local work, only the upper bound follows. Zero combination work can leave linear leaf cost alone.

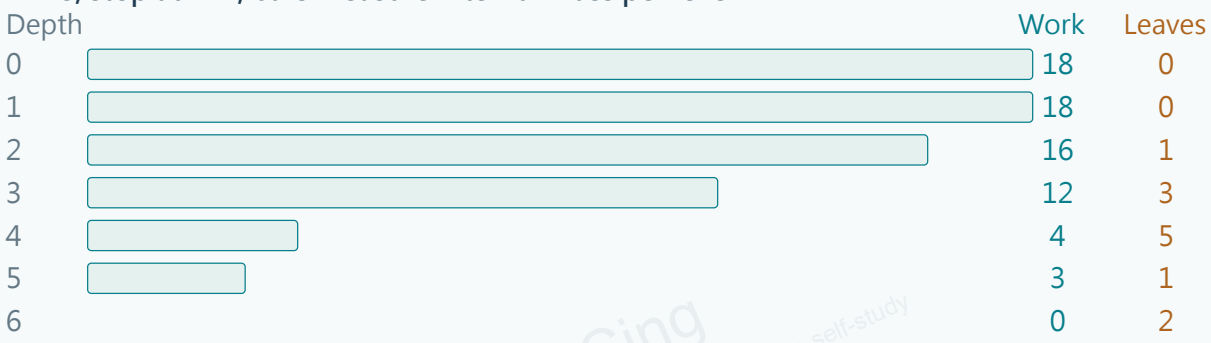
Unequal splits: the complete call tree



Each edge is one executed call. Orange leaves stop at different depths; the deepest path is 18 →12 →8 →6 →4 →3 →2.

Unequal splits: distinguish complete and truncated levels

n=18; stop at n≤2; bars measure internal mass per level



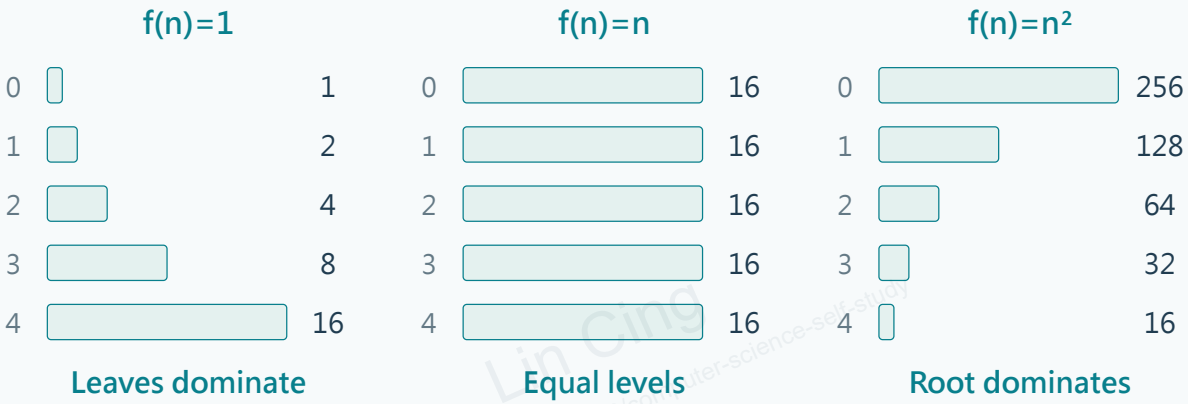
Charge leaves separately; stopped nodes produce no descendants.

The figure stops at $n \leq 2$; larger nodes split into $\lfloor n/3 \rfloor$ and $n - \lfloor n/3 \rfloor$. Complete levels initially have mass 18. After leaves appear, **only internal nodes** pay combination work.

The shortest path limits the complete levels usable for a lower bound. The longest path controls the depth needed for an upper bound. Multiplying the deepest height by n does not establish a lower bound for truncated levels.

Read the level work behind the three cases — diagram

Independent scale per panel; labels are total cost per level



Read the level work behind the three cases

All three use $n = 16, a = b = 2, T(1) = 1$. Nonrecursive terms are $1, n, n^2$. Costs by level, including leaves:

$f(n)$	Depths 0,1,2,3,4	Total
1	1,2,4,8,16	31
n	16,16,16,16,16	80
n^2	256,128,64,32,16	496

These illustrate leaf, equal-level, and root domination. The theorem still requires fixed parameters, polynomial gaps, and case 3 regularity. A drawing alone does not verify those conditions.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Master theorem: check the hypotheses

Let fixed $a \geq 1, b > 1$ and nonnegative local work f define

$$T(n) = aT(n/b) + f(n), \qquad p = \log_b a.$$

Base costs on a constant-size interval have fixed positive upper and lower bounds. Start on exact powers; rounding conditions follow later. An actual call count a is integral; a real a can instead be a recurrence weight.

Case	Required hypotheses	Result
1	Fixed $\epsilon > 0$; $f(n) = O(n^{p-\epsilon})$	$\Theta(n^p)$
2	$f(n) = \Theta(n^p)$	$\Theta(n^p \lg n)$
3	Fixed $\epsilon > 0$; $f(n) = \Omega(n^{p+\epsilon})$; eventually $af(n/b) \leq cf(n)$ for fixed $c < 1$	$\Theta(f(n))$

Merely smaller or larger than n^p is insufficient. Cases 1 and 3 require a fixed polynomial gap; case 3 also requires regularity.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete case classification examples

Every row assumes fixed positive base costs. Compute $p = \log_b a$ rather than guessing it from a plot.

$(a, b, f(n))$	Concrete hypothesis check	Tight bound
$(9, 3, n)$	$p = 2$; choose $\epsilon = 1$	$\Theta(n^2)$
$(1, 3/2, 1)$	$p = 0$; $f(n) = \Theta(n^0)$	$\Theta(\lg n)$
$(3, 4, n \lg n)$	$p \approx 0.7925$; choose $\epsilon = 0.1$; regularity ratio at most $3/4$	$\Theta(n \lg n)$
$(8, 2, n^2)$	$p = 3$; choose $\epsilon = 1$	$\Theta(n^3)$
$(7, 2, n^2)$	$p = \lg 7$; choose $\epsilon = \lg 7 - 2 > 0$	$\Theta(n^{\lg 7})$

For the third row,

$$\frac{3f(n/4)}{f(n)} = \frac{3}{4}\frac{\lg n - 2}{\lg n} \le \frac{3}{4} \quad (n \ge 4).$$

By contrast, $2T(n/2) + n \lg n$ has no fixed polynomial gap. None of the classical three cases applies directly, but the level sums next still solve it.

Sum logarithmic boundary cases

Let $n = b^L$, $p = \log_b a$, and $f(n) = \Theta(n^p (\lg n)^k)$ for fixed real k. Above a fixed base interval, renumber depth j by distance $r = L - j$ from the leaf end. Level cost is $\Theta(n^p r^k)$.

Add $\Theta(n^p)$ leaf work to n^p times the resulting power sum:

$$T(n) = \begin{cases} \Theta(n^p (\lg n)^{k+1}) & k > -1, \\ \Theta(n^p \lg \lg n) & k = -1, \\ \Theta(n^p) & k < -1. \end{cases}$$

For $k > -1$, integral comparison gives $\sum_{r=1}^{L} r^k = \Theta(L^{k+1})$. At $k = -1$ it is harmonic; for $k < -1$ it converges to a positive constant. For negative k, apply the formula only above a fixed base interval larger than 1, avoiding negative powers of $\lg 1$.

Thus $2T(n/2) + n/\lg n$ gives $\Theta(n \lg \lg n)$; replacing its local term by $n/(\lg n)^2$ gives $\Theta(n)$. These follow from level sums, not classical case 1.

A counterexample without regularity

Restrict to $n = 2^L$, with $T(1) = 1$ and

$$f(n) = \begin{cases} n^2 & L \text{ even}, \\ n^3 & L \text{ odd}, \end{cases} \qquad T(n) = 2T(n/2) + f(n).$$

Always $f(n) \geq n^2$, giving a fixed polynomial gap above $n^p = n$. For even $L \geq 2$, however, the child exponent is odd:

$$T(n) \geq 2f(n/2) = n^3/4, \qquad \frac{T(n)}{f(n)} \geq n/4 \to \infty.$$

Hence $T(n) \notin \Theta(f(n))$. The regularity ratio itself equals $n/4$, so no common $c < 1$ works eventually.

This illustrates omission of a hypothesis, not failure of the theorem. For an oscillating function, a bounded change in its argument need not produce comparable values.

Why the master theorem holds

Start with $n = b^L, T(1) = t > 0$. Expanding L times gives

$$T(n) = tn^p + \sum_{j=0}^{L-1} a^j f(n/b^j).$$

Depth j has total weight a^j; when a counts actual calls, it is also the node count. The leaf term tn^p gives a positive lower bound in every case.

In case 1, eventually $f(x) \leq Cx^{p-\epsilon}$. Large-size levels contribute at most

$$Cn^{p-\epsilon} \sum_{j=0}^{L-1} (b^\epsilon)^j = O(n^{p-\epsilon} b^{\epsilon L}) = O(n^p).$$

The ratio $b^\epsilon > 1$ makes this sum increase toward the leaves. Extending the bound across finitely many small arguments adds at most $O(n^p)$: their total weight has that order, and their fixed function values are absorbed into constants.

Adding leaves gives $O(n^p)$; their positive cost supplies $\Omega(n^p)$, proving $\Theta(n^p)$.

Equal-level and root-dominated proofs

Case 2. Since $a = b^p$, every sufficiently large-size level costs

$$a^j \Theta((n/b^j)^p) = \Theta(n^p).$$

There are $L - O(1) = \Theta(\lg n)$ such levels. Summing their upper and lower bounds gives $\Theta(n^p \lg n)$ and covers leaf work.

Case 3. Repeated regularity gives, while arguments remain sufficiently large,

$$a^j f(n/b^j) \le c^j f(n), \qquad 0 < c < 1.$$

Their sum is at most $f(n)/(1-c)$. Remaining fixed-size levels and leaves cost $O(n^p)$, which is $O(f(n))$ by polynomial separation. The root's local work $f(n)$ supplies the matching lower bound, proving $\Theta(f(n))$.

Small-input work is not constant across the entire tree. There can be $\Theta(n^p)$ small nodes; their total must be included.

Rounding errors remain bounded

Along an actual call path, let $q_0 = n$ and let q_{j+1} be either $\lfloor q_j/b \rfloor$ or $\lceil q_j/b \rceil$. Both satisfy $|q_{j+1} - q_j/b| \le 1$. For fixed $C \ge b/(b-1)$, induction gives

$$|q_j - n/b^j| \le C,$$

because the next error is at most $C/b + 1 \le C$. It does not accumulate linearly with depth.

Choose a fixed base interval $q \le n_0$, with $n_0 > C + 2$ and large enough for strict shrinking. The two error bounds place every leaf at depth $\log_b n + O(1)$. Where $n/b^j \ge 2C$, actual sizes and unrounded sizes are within fixed factors.

With an integral number a of actual children, leaf depths differ by only a constant, giving $\Theta(a^{\log_b n}) = \Theta(n^p)$ leaves and $O(n^p)$ total fixed-size work. For $a = 1$, the tree is one path with constant bottom work.

Conditions for all integer sizes

Here a is an integral number of actual subcalls. Real weights on exact powers were handled by the earlier expansion.

Cases 1 and 2: Actual sizes q_j and n/b^j are within fixed factors on large-size levels. Even for a negative fixed exponent,

$$q_j^{p-\epsilon} = \Theta((n/b^j)^{p-\epsilon}).$$

Case 1 therefore retains its polynomial bound and geometric sum. In case 2, each complete large-size level still costs $\Theta(n^p)$, with $\Theta(\lg n)$ levels. Charge incomplete fixed-size levels separately as $O(n^p)$. The logarithmic extension also survives: away from the fixed base interval, $\log q_j$ is comparable to distance from the bottom.

Case 3: Apply regularity to actual rounded arguments. A sufficient condition is

$$a \max\{f(\lfloor x/b \rfloor), f(\lceil x/b \rceil)\} \le cf(x), \qquad c < 1.$$

Then level work contracts by c, and polynomial separation absorbs bottom work. Bounded argument error alone does not justify dropping this condition for arbitrary f.

Regularity can imply a polynomial gap

On exact powers, suppose $af(x/b) \le cf(x)$ for fixed $0 < c < 1$, and $f(x_0) > 0$ at a fixed bottom point. Reversing the inequality through k levels gives

$$f(b^k x_0) \ge (a/c)^k f(x_0).$$

Since $\log_b(a/c) = p + \log_b(1/c)$, set $\epsilon = \log_b(1/c) > 0$. Along this sequence, $f(n) = \Omega(n^{p+\epsilon})$.

This explains why some versions omit a separate polynomial-gap hypothesis under suitable positive bottom conditions. We retain the complete three-case assumptions for direct checking.

The argument covers $n = b^k x_0$. Extending it to every real or integer input requires control of the final base interval and a positive lower bound for f there. A proof on one sequence alone does not cover all inputs.

Maximum nonempty contiguous subarray

Given $n \geq 1$ finite real values $A[1..n]$, find a nonempty contiguous interval $[i, j]$ maximizing

$$\sum_{k=i}^{j} A[k],$$

and return (i, j, sum). Any maximizing interval is acceptable. We count exact additions and comparisons; floating-point error, NaN, and overflow are outside this model.

For prices $P_0, \ldots, P_n$, let $A_i = P_i - P_{i-1}$. Then $P_j - P_{i-1} = \sum_{k=i}^{j} A_k$: interval $i..j$ means buying on day $i - 1$ and selling on day j.

Allowing no transaction adds an empty interval with value 0 and changes all-negative cases. Enumerating each start and extending its end with a running sum takes $\Theta(n^2)$ time and $O(1)$ auxiliary space. Re-summing every interval instead takes $\Theta(n^3)$.

Crossing means suffix plus prefix

For $low < high$, set $mid = low + \lfloor (high - low)/2 \rfloor$. Every nonempty interval belongs to exactly one class:

1. Entirely in $low..mid$.
2. Entirely in $mid + 1..high$.
3. Crossing the boundary, with $i \leq mid < j$.

A crossing interval contains both mid and $mid + 1$. It splits uniquely into a left suffix $i..mid$ and a right prefix $mid + 1..j$.

The two endpoints can be chosen independently. Thus its best sum is **the best nonempty left suffix plus the best nonempty right prefix**. Scan both sides separately rather than enumerating endpoint pairs. The best subarray within each half need not touch the boundary, so adding those two answers is invalid.

Crossing means suffix plus prefix — diagram

mid=5: a crossing interval includes positions 5 and 6

Left sums:−1, 3, 0, 1,−1 → best 3, start 4

Right sums:2, 3,−2, 2 → best 3, end 7

Combine A[4..7]=[4,−1, 2, 1], sum 6.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete crossing scan

```
CROSS(A, low, mid, high):
    left = -infinity; total = 0
    for i = mid downto low:
        total += A[i]
        if total > left: left = total; best_i = i
    right = -infinity; total = 0
    for j = mid+1 to high:
        total += A[j]
        if total > right: right = total; best_j = j
    return (best_i, best_j, left+right)
```

After each left-loop update, `total` is the sum of $A[i..mid]$, and `left` is the maximum among all left suffixes examined so far. The initial $-\infty$ is a comparison marker, not input data; the first finite candidate updates it and sets an endpoint.

Each iteration adds exactly one candidate and takes the maximum, preserving the invariant. All left suffixes are covered at termination; the right prefixes are analogous. Adding the two maxima gives the best crossing sum. Total scan iterations are

$$(mid - low + 1) + (high - mid) = high - low + 1.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursive maximum subarray and proof

```
MAX-SUBARRAY(A, low, high):
    if low == high: return (low, high, A[low])
    mid = low + floor((high-low)/2)
    L = MAX-SUBARRAY(A, low, mid)
    R = MAX-SUBARRAY(A, mid+1, high)
    C = CROSS(A, low, mid, high)
    return a tuple with largest sum among L, R, C
```

Use strong induction on interval length. A singleton is its only answer. For a longer interval, both children are nonempty and smaller. Their contracts solve the first two classes; CROSS solves the third. The classes are disjoint and exhaustive, so taking their maximum is correct. Strict shrinking also guarantees termination.

Operation cost is

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) = \Theta(n \lg n).$$

Sequential subcalls with index arguments retain only constant many indices and sums per frame, giving $O(\lg n)$ peak stack space. Copying subarrays changes allocation and lifetime costs and must be analyzed separately.

Nine-element crossing trace

For $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$, use $low = 1, high = 9, mid = 5$.

Left index i	Sum of $A[i..5]$	Best sum and start so far
5	-1	-1,5
4	3	3,4
3	0	3,4
2	1	3,4
1	-1	3,4

Right-prefix sums at $j = 6, 7, 8, 9$ are $2, 3, -2, 2$, with best interval $6..7$ of sum 3. The crossing answer is $4..7$ of sum 6. Left-only and right-only maxima are $4..4$ and $9..9$, both of sum 4. Return $(4, 7, 6)$.

For all-negative input $[-8, -2, -5]$, the nonempty contract returns $(2, 2, -2)$, not 0.

A linear algorithm's state and invariant

Let E_i be the best nonempty sum ending exactly at i, and B_i the best nonempty sum within $1..i$:

$$E_1 = B_1 = A_1, \qquad E_i = \max(A_i, E_{i-1} + A_i), \quad B_i = \max(B_{i-1}, E_i).$$

An interval ending at i either consists only of A_i or extends one ending at $i-1$. In the latter case, the best previous ending sum suffices. This classification and the bases prove both state meanings by induction.

Choosing A_i resets the start to i; extending preserves the old start. Save that start and endpoint i when the global best improves. Constant work per iteration gives $\Theta(n)$ time and $O(1)$ auxiliary space.

For the nine-value example, $E_i = [-2, 1, -2, 4, 3, 5, 6, 1, 5]$ and $B_i = [-2, 1, 1, 4, 4, 5, 6, 6, 6]$. Divide-and-conquer is not the fastest solution here; it illustrates a different classification and combination proof.

From scalar products to block multiplication

Standard multiplication computes $C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$. After step k, the accumulator contains the first k products. The n^2 outputs each take n multiply-add steps: $\Theta(n^3)$ operations for this algorithm, not a universal lower bound.

For even n, use $n/2$-order blocks:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad B = \begin{bmatrix} e & f \\ g & h \end{bmatrix}.$$

Their product satisfies

$$C_{11} = ae + bg, \quad C_{12} = af + bh, \quad C_{21} = ce + dg, \quad C_{22} = cf + dh.$$

Preserve multiplication order; matrix blocks generally do not commute.

Full cost of eight recursive products

Pad to a power-of-two order, then extract the original-size result. Each call reads its own view's order rather than reusing the original order.

```
BLOCK-MULTIPLY(A,B):
    n = A.rows
    if n == 1: return [[A[1,1]*B[1,1]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    C11 = BLOCK-MULTIPLY(a,e)+BLOCK-MULTIPLY(b,g)
    C12 = BLOCK-MULTIPLY(a,f)+BLOCK-MULTIPLY(b,h)
    C21 = BLOCK-MULTIPLY(c,e)+BLOCK-MULTIPLY(d,g)
    C22 = BLOCK-MULTIPLY(c,f)+BLOCK-MULTIPLY(d,h)
    return the four blocks assembled into C
```

An index view retains an origin, stride, and order, permitting constant-time splitting. Explicit block copies cost $\Theta(n^2)$, the same order as fixed many block additions. Hence $T(n) = 8T(n/2) + \Theta(n^2)$ with a $\Theta(1)$ base.

Eight recursive products give 8^j calls at depth j and $8^{\lg n} = n^3$ leaves. The branch count changes the exponent; it cannot be hidden in one child's constant.

All seven Strassen products

Compute each product recursively with STRASSEN. Additions and subtractions inside parentheses are elementwise block operations.

$$\begin{aligned} M_1 &= (a+d)(e+h), & M_2 &= (c+d)e, \\ M_3 &= a(f-h), & M_4 &= d(g-e), \\ M_5 &= (a+b)h, & M_6 &= (c-a)(e+f), \\ M_7 &= (b-d)(g+h). \end{aligned}$$

Assemble the four output blocks:

$$\begin{aligned} C_{11} &= M_1 + M_4 - M_5 + M_7, & C_{12} &= M_3 + M_5, \\ C_{21} &= M_2 + M_4, & C_{22} &= M_1 - M_2 + M_3 + M_6. \end{aligned}$$

For $n = 1$, multiply the two scalars. For $n > 1$, split, compute these seven products, and assemble the four blocks. Together with the base case, these steps specify the recursive procedure.

Correctness of all four output blocks

Use distributivity and additive cancellation, preserving every product's factor order:

$$\begin{aligned}
C_{11} &= (ae + ah + de + dh) + (dg - de) \\
&\quad - (ah + bh) + (bg + bh - dg - dh) = ae + bg, \\
C_{12} &= (af - ah) + (ah + bh) = af + bh, \\
C_{21} &= (ce + de) + (dg - de) = ce + dg, \\
C_{22} &= (ae + ah + de + dh) - (ce + de) \\
&\quad + (af - ah) + (ce + cf - ae - af) = cf + dh.
\end{aligned}$$

Scalar multiplication is a correct base. If the seven smaller products are correct by induction, these identities assemble all four blocks of AB, completing the proof.

The identities require addition, subtraction, and distributivity. They do not directly apply to a general semiring without subtraction.

Numerical products and reconstruction

Take $A = \begin{bmatrix} 2 & -1 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 2 \\ -2 & 1 \end{bmatrix}$.

Product	Substitution	Value
M_1	$(2 + 4)(5 + 1)$	36
M_2	$(3 + 4)5$	35
M_3	$2(2 - 1)$	2
M_4	$4(-2 - 5)$	-28
M_5	$(2 - 1)1$	1
M_6	$(3 - 2)(5 + 2)$	7
M_7	$(-1 - 4)(-2 + 1)$	5

The four blocks are $36 - 28 - 1 + 5 = 12$, $2 + 1 = 3$, $35 - 28 = 7$, and $36 - 35 + 2 + 7 = 10$. Direct multiplication verifies:

$$AB = \begin{bmatrix} 10 + 2 & 4 - 1 \\ 15 - 8 & 6 + 4 \end{bmatrix} = \begin{bmatrix} 12 & 3 \\ 7 & 10 \end{bmatrix}.$$

Numerical products and reconstruction — diagram

Keep C11,C12,C21,C22 in their correct block positions

A			B			C	
2	-1	×	5	2	=	12	3
3	4		-2	1		7	10

C11: 36−28−1+5=12

C12: 2+1=3

C21: 35−28=7

C22: 36−35+2+7=10

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Time, padding, and memory

The seven Strassen subproducts need ten input-block additions/subtractions and eight more to assemble the outputs. View splitting is $O(1)$, giving

$$T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\lg 7}), \qquad \lg 7 \approx 2.80735.$$

Larger addition constants may make small matrices slower. A fixed cutoff to standard multiplication preserves the exponent. Algebraic correctness alone gives no floating-point error guarantee.

For arbitrary $n \geq 1$, pad to $m = 2^{\lceil \lg n \rceil}$, where $n \leq m < 2n$. The top-left $n \times n$ result is the original product, and $m^{\lg 7} < 7n^{\lg 7}$ preserves the order. Return an empty matrix directly at zero size.

Execute subcalls sequentially and release temporaries promptly. Retaining only fixed many current-level blocks gives peak space

$$S(n) \leq S(n/2) + O(n^2) = O(n^2),$$

plus $O(\lg n)$ frames. Simultaneously retaining every child workspace requires a different space recurrence.

Extension: conditions for Akra–Bazzi

We cite the theorem in MIT's *Mathematics for Computer Science* (2010), § 10.4.3, using positive bounded base costs as sufficient conditions. **Applications below are not a full proof of this theorem.**

$$T(x) = \sum_{i=1}^{k} a_i T(b_i x + h_i(x)) + g(x), \qquad x > x_0.$$

- Fixed finite k, fixed $a_i > 0$, and fixed $0 < b_i < 1$.
- Fixed $0 < d_1 \le T(x) \le d_2$ on $0 \le x \le x_0$.
- Nonnegative differentiable g, with $|g'(x)|$ polynomially bounded; $|h_i(x)| = O(x/\log^2 x)$.
- Choose x_0 so every child lies in $[0, \rho x]$ for a fixed $\rho < 1$, making recursion well-defined and terminating.

These are the conditions used here. Zero base costs and identically zero g would give identically zero cost, invalidating a conclusion with a positive leaf term.

Find the unique exponent before integrating

Let p satisfy $\sum_i a_i b_i^p = 1$. The cited theorem gives

$$T(x) = \Theta\left(x^p \left[1 + \int_1^x \frac{g(u)}{u^{p+1}}\, du\right]\right).$$

Since $0 < b_i < 1$, $F(p) = \sum_i a_i b_i^p$ is continuous and strictly decreasing, tending to infinity as $p \to -\infty$ and to zero as $p \to +\infty$. The real root is unique.

For $T(x) = T(x/3) + T(2x/3) + x$, with positive constant base costs, the equation $(1/3)^p + (2/3)^p = 1$ gives $p = 1$. Therefore

$$T(x) = \Theta\left(x \left[1 + \int_1^x \frac{du}{u}\right]\right) = \Theta(x \log x).$$

This agrees with the earlier complete-level and maximum-depth bounds. Keep the assumptions of both arguments explicit.

Rounding and a changed ratio are different

Floor, ceiling, and fixed shifts have bounded additive error, satisfying $O(x/\log^2 x)$. Choose a sufficiently large base interval to ensure shrinking.

In $T(n) = T(n/2 + n/4) + T(n/2) + n$, however, the first child is $3n/4$. The term $n/4$ is not a small perturbation; two one-half ratios give the wrong equation.

The correct equation is $(3/4)^p + (1/2)^p = 1$, with unique root $p \approx 1.507126591639 > 1$. Since

$$\int_1^n u^{-p}\,du = \frac{1 - n^{1-p}}{p-1} = \Theta(1),$$

we obtain $T(n) = \Theta(n^p)$. The equation defines the exact exponent; the decimal is approximate.

If another source writes child sizes as n/c_i, its parameters correspond to $b_i = 1/c_i$ here. Convert the equation together with the notation.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Further reading and theorem sources

- Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms*, 3rd Edition: Chapter 2 for merge sort; Chapter 4 for divide-and-conquer, substitution, recursion trees, and the master theorem with its proof.
- Demaine and Leiserson, MIT 6.046J (Fall 2005), Lectures 2–3: recurrences and their connection to binary search, powering, Fibonacci, and matrix multiplication.
- Lehman, Leighton, Meyer, *Mathematics for Computer Science* (2010), § 10.4.3, Theorem 10.4.1: official MIT notes. We use positive bounded base costs as sufficient conditions.

The chapter proves the three master-theorem cases. Akra–Bazzi is a cited extension; its applications verify hypotheses, solve the exponent equation, and evaluate the integral.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study